

第 7 章

验收测试

验收测试(Acceptance Test)是在软件产品完成了功能测试和系统测试之后、产品发布之前所进行的软件测试活动。它是技术测试的最后一个阶段,也称为交付测试。通过了验收测试,产品就会进入发布阶段。验收测试一般根据产品规格说明书,严格检查产品,逐行逐字地对照说明书上对软件产品所做出的各方面要求,确保所开发的软件产品符合用户预期的各项要求,即验收测试是检验产品和产品规格说明书(包括软件开发的技术合同)的一致性。同时,验收测试,也可以称为现场测试,一般在实际的运行环境或用户的使用环境下进行,而且用户也参与到验收测试过程中。通过验收测试,也就是通过了用户的验收,说明软件产品的质量达到了客户的要求,测试项目到此告一段落。

7.1 验收测试过程

验收测试是在系统测试之后进行,所以验收测试的前提条件是系统或软件产品已通过了内部测试,即从软件开发组织来看,所有要做的测试都已完成,所发现的严重缺陷都已修正。然后,和用户一起来验收软件系统,在真实的环境下运行软件系统,看看是否存在和用户要求不一致的问题,或违背产品规格书的要求。另外,测试人员不可能完全预见用户实际使用程序的情况,也就不可能发现所有的错误。例如,用户可能错误地理解命令、提供一些奇怪的数据组合或可能对输出信息迷惑不解等。因此,软件是否真正满足最终用户的要求,应由用户进行一系列“验收测试”。

在产品发布前,与用户共同完成验收测试的可能性就比较小,往往会转变为另两种测试—— α 测试和 β 测试。从这种意义上看,验收测试就不存在,或者说,我们不得不将“验收测试”扩展为广义的概念——在软件产品完成了功能测试和系统测试之后,在产品发布之前所进行的软件测试活动。所以,如果是公司自行开发的产品,可以由测试人员和产品设计部门、市场部门等共同进行,可能还包括技术支持、产品、培训等部门人员。

在敏捷测试流程(如 Scrum)中也有验收测试,和传统的验收测试概

念也不一样,可以看作是全面的系统测试或回归测试,因为之前是持续测试,一方面主要是单元测试和集成测试,侧重各个功能特性(或用户故事)的测试,对整个系统测试不够;另一方面,持续测试比较零碎,版本不断更新,完成一点代码做一点测试,需要系统的回归测试。

验收测试既可以是非正式的测试,也可以是有计划、系统的测试。验收测试是验证系统是否达到了用户需求规格说明书(可能包括项目或产品验收准则)中的要求,试图尽可能地发现软件中存留的缺陷,从而为软件进一步改善提供帮助,并保证系统或软件产品最终被用户接受。验收测试主要包括易用性测试、兼容性测试、安装测试、文档(如用户手册、操作手册等)测试等几个方面的内容。

1. 测试步骤

(1) 在需求分析阶段建立测试计划,了解软件功能和性能要求、软硬件环境要求等,并特别要了解软件的质量要求和具体的验收要求。根据软件需求和验收要求编制验收测试计划,制定需测试的测试项,制定测试策略及验收通过准则,并让客户参与计划评审,直至通过。

(2) 建立测试环境,根据验收测试计划、项目或产品验收准则完成测试用例的设计,并经过评审。

(3) 准备测试数据,执行测试用例,记录测试结果。

(4) 分析测试结果,根据验收通过准则分析测试结果,做出验收是否通过及测试评价。验收结果通常会有以下4种情况。

① 测试项目通过验收;

② 测试项目没有通过验收,但问题不大,双方沟通可以达成一个补充协议,在维护后期或下一个版本改进;

③ 测试项目没有通过验收,并且不存在变通方法,需要很大的修改;

④ 测试项目无法评估或者无法给出完整的评估结果,此时必须找到其深层次的原因。如果是该测试项目没有说明清楚,应该修改验收测试计划。

(5) 提交测试报告。根据产品设计说明书、详细设计说明书、验收测试结果和发现的错误信息,评价系统的设计与实现,最终通过验收测试报告给予完整的、详细的描述。

2. 通过标准

(1) 完全执行了验收测试计划中的每个测试用例。

(2) 在验收测试中发现的错误已经得到修改并且通过了测试或者经过评估留到下一版本中修改。

(3) 完成软件验收测试报告。

3. 注意事项

(1) 必须编写正式的、单独的验收测试计划。该计划中必须有明确的验收标准。通过验收测试的产品并不表示它不存在缺陷。有时为了争夺市场,允许非严重性的缺陷存在,但是,不能根据测试结果再来修改验收标准。

(2) 验收测试必须在实际运行环境中或尽可能模拟实际的环境中进行。测试环境与实际运行环境的差异可能会遗漏少数的严重缺陷,这样会严重影响用户的满意度。例如,一个数据库查询系统,在拥有少量数据时工作正常,而在实际环境中,可能拥有海量的数据,仅仅简单的查询操作就可能耗费大量系统资源,丧失可用性。

(3) 验收测试一般需要由用户和测试部门共同完成,如果不是用户委托的项目,而是软件

公司自行研发的产品,就不会有用户参与测试,而是让产品设计部门、市场部门、技术支持等参与测试,或者让公司所有员工参与产品的试用。

α 测试和 β 测试

α 测试是指软件开发公司组织内部人员模拟各类用户对即将面市软件产品(称为 α 版本)进行测试,试图发现错误并修正。 α 测试的关键在于尽可能逼真地模拟实际运行环境和用户对软件产品的操作并尽最大努力涵盖所有可能的用户操作方式。经过 α 测试调整的软件产品称为 β 版本。紧随其后的 β 测试是指软件开发公司组织各方面的典型用户在日常工作中实际使用 β 版本,并要求用户报告异常情况、提出批评意见。然后软件开发公司再对 β 版本进行改错和完善。

7.2 产品规格说明书的验证

产品规格说明书,也称功能规格说明书(Functional Specification),是基于需求的定义,详细描述将要开发出一个什么样的产品,包括产品的用途、有哪些功能、用户界面的表现形式及其交互特性等。它通常会遵守公司内部约定的模板或其他要求,采用 Word、PDF、Visio 或 HTML 等文档格式,包括文字、表格、图形甚至动画等内容。

7.2.1 产品规格说明书的评审

产品规格说明书是测试的标准,如果没有它,谁都不清楚产品应该是什么样,最终会是什么样,会让测试工作无从着手,陷入无尽的困难之中。产品说明书的特性决定了对其复审(Review)的重要性。经验显示,充分的复审能排除约 60% 的错误,在产品的设计阶段,发现需求和设计上大部分的缺陷,避免返工,可为项目节省大量的成本。

产品规格说明书的复审不是验收测试阶段的主要任务,它应该在整个产品生命周期的初期,即在产品规格说明书初稿完成后就开始评审,在编程之前完成评审。软件评审方法包括互为复审或称同行评审(Peer-to-Peer Review)、走查(Walkthrough)和正式会议审查(Inspection)。产品规格说明书的评审,先可以采用邮件分发审查方法,通过邮件将产品规格说明书分发下去,收集大家的反馈意见,进行讨论和修改。在后期,可以集中采用会议评审的方式,尽可能发现潜在的工作问题,阐释各种疑问,力求大家达成一致意见。

会议审查是一种系统化、严密的集体评审方法,其过程一般包含制定计划、准备和组织会议、跟踪和分析结果等。在会议评审过程中涉及多个角色,如评审组长、作者、评审人员、列席人员和会议记录人员等。在评审过程中,要注意以下几点。

- (1) 从客户的角度和立场进行审核工作。
- (2) 检验套用标准的正确性,不要和行业规范相抵触。
- (3) 审查、研究同类产品。
- (4) 验证产品规格说明书的完整性、准确性、一致性、合理性等特性。

7.2.2 产品规格说明书的验证

产品规格说明书的验证大部分属功能性测试的范畴。早在集成测试完成后、系统测试开始前,就开始对软件系统进行功能性测试。但早期的功能性测试旨在验证产品说明书定义的

功能实现以保证后续测试的进行,并尽早地发现问题。

在验收测试阶段,产品规格说明书的验证将更加严格、彻底地进行。大多数情况下,测试人员并不能得到完美的产品规格说明书,尽管它已经通过了审核,但由于评审不够充分、需求的变更、设计修改等影响,都会导致产品规格说明书不得不做相应更改。所以测试人员不仅要根据产品说明书的每一个特性导出测试用例,而且应针对上述变动,及时更新测试用例,确保产品规格说明书和测试用例保持一致。

测试人员应根据产品说明书,逐字逐句地验证产品的每一项特性,并在验证结束时提交基于产品规格说明书的验证报告。该报告可以是正式的,也可以是非正式的,其目的在于能够在验收测试初期进行评判软件特性是否都已实现、是否可以进行全面的验收测试。验证报告的内容应包括所有特性的清单。

- (1) 已经实现的特性标识为通过。
- (2) 特性没有实现的报告缺陷并在报告中体现。
- (3) 特性基本实现,但与产品说明书中内容不相一致。报告缺陷并在报告中体现。
- (4) 特性基本实现,但存在一些问题或错误。

7.2.3 文档的测试

软件文档是软件的重要组成部分,文档的错误也是软件缺陷。错误的解释可能会引导用户无法完成某些软件已具有的功能,例如,安装文档不正确,用户无法进行安装,是一个很严重的缺陷。从另一方面看,用户通过文档可以掌握具体的使用方法,提高了易用性;避免了用户在摸索使用中的一些不可预期的操作,也就相对避免了一些不可预期的错误的发生,从而提高了产品的可靠性。当用户在遇到问题时,多数会向朋友或同事询问解决方法,再就是通过帮助文档或请求公司帮助。约 30% 的用户通过文档解决了问题,也就避免了公司提供费用不菲的技术支持。综上所述,文档测试也是不可忽视的,包括产品规格说明书的验证和后续的安装文档的测试。

1. 文档的种类

(1) 联机帮助文档或用户手册。这是人们最容易想到的文档。用户手册是指导用户如何使用软件的文档,可能是一本书,也可能仅仅是 PDF 或 chm 格式的文件。有时以联机帮助文档的形式出现,具有索引和搜索功能,用户可以方便快捷地查找所需信息。Microsoft Word 的联机帮助文档如图 7-1 所示。

(2) 指南和向导。是程序和文档融合在一起形成的、可以引导用户一步一步完成任务的一种工具,如 Microsoft Office 助手。

(3) 安装、设置指南。简单的安装文档可以是一页纸,复杂的指南可以是一本手册。

(4) 示例及模板。例如,某些系统提供给用户填写的表单模板。

(5) 错误提示信息。常常被忽略,但确属于文档。举一个较特殊的例子,服务器系统运行时检测到系统资源达到临界值,或受到攻击,给管理员发送警告邮件。

(6) 用于演示的图像和声音。

(7) 授权/注册登记表及用户许可协议。

(8) 软件的包装、广告宣传材料。有些用户会认真对待,并很好地利用它,若出现错误或缺少必要的信息可能带来麻烦,甚至标签上的信息等均为文档测试的内容。



图 7-1 Microsoft Word 的联机帮助

2. 怎样进行文档测试

很多程序员能编写出好程序,却写不出清晰的文档,技术手册的质量不够高,测试的难度更高,需要事先仔细阅读,并与文档作者进行充分交流,尝试每个示例,就能发现其中的问题。文档测试主要检查文档的正确性、完备性、易理解性和一致性。

(1) 正确性是指不要把软件的功能和操作写错,也不允许文档内容前后矛盾。测试描述功能的陈述是否必要?有没有多余信息?功能是否满足的客户要求?

(2) 完备性是指文档不可以“虎头蛇尾”,更不能漏掉关键内容,检查是否有遗漏和丢失的内容?文档中很多内容对开发者可能是“显然”的,但对用户而言不见得都是“显然”的。

(3) 易理解性。文档不含糊,清晰,要让大众用户看得懂,容易理解。例如,内容描述是否一清二楚?术语、缩写用户是否理解?内容和主题是否一致?

(4) 一致性,产品功能描述是否自相矛盾?与其他功能有没有冲突?

7.3 用户界面和可用性测试

用于软件程序交互的方式称为用户界面(User Interface, UI)。与早期的软件相比,现在使用的个人计算机都有复杂的图形用户界面(Graphical User Interface, GUI)。虽然 UI 各不相同,但其本质是相同的,都是提供用户与计算机之间交互和交流的桥梁。

许多产品都应用人体工程学的研究成果,使产品更具人性化,使用时更加灵活、舒适。软件产品也是一样,应以软件的最终使用者——客户为出发点。好的用户界面包括 7 个要素:符合标准和规范、直观性、一致性、灵活性、舒适性、正确性、实用性。

1. 符合标准和规范

在现有的平台上软件都遵守一定的标准和规范,Windows、Mac OS 等操作系统都有自己的界面标准,这些标准都是经过多年实践的积累而形成的,已经得到用户的接受,例如,软件应

该有什么样的外观、何时使用复选框、何时使用单选按钮、何时使用提示信息、警告信息或者严重警告信息等,形成人们认可的惯例,如图 7-2 所示。



图 7-2 Windows 的三种信息使用的方式

由于多数用户已经熟悉并接受了这些标准和规范、或已经认同了这些信息所代表的意义。这时,如果用“提示信息”代替“严重警告”,很难引起用户的重视,他可能随手关闭,造成严重后果后,用户自己可能还不知道,自然得不到用户的认同。测试人员就应该将此类问题报告为缺陷。如果软件在某一个平台上运行,如同产品规格说明书一样,该平台的标准和规范也应作为测试的依据。如果正在测试的软件本身就是一个软件平台,那么软件设计者就应创立一套标准,贯穿于整个软件的设计开发过程,保持软件与行业标准、规范或约定相一致。

2. 直观性

考虑用户界面的直观性,首先确定功能操作界面、提示或期待的结果是否直观、显著,并出现在预期的地方或时间。例如,执行结果已经显示出来,但因其不明显,客户使用时还在焦急地等待结果的出现。再比如,软件中某个图标用了软件编程中常用的术语缩写,开发人员和测试人员往往因为熟悉而忽略,而用户就很难理解其含义,只能猜测。其次,考虑用户界面的组织和布局是否合理,界面是否洁净、不拥挤以及是否有多余的功能,是否太复杂难以掌握等因素。有一个设计展示网站(www.jaspermorrison.com),如图 7-3 所示,其主界面非常直观,各类设计的链接都是通过直观的图形描述,而且整个页面没有任何多余的内容。



图 7-3 直观页面的示例

3. 一致性

包括软件本身的一致性,以及与公司其他软件、第三方软件的一致性。字体是否一致、界

面的各元素风格是否一致是比较容易判定的。另外,一致性的问题通常还体现在平台的标准和规范上,用户习惯于将某一程序的操作方式带到另一个程序中使用。例如,在 Windows 平台客户已经习惯用 Ctrl+C 键表示复制操作,而在软件中将复制操作的快捷键定义为其键必定会给用户造成挫败感,难以接受。如果在同一软件中不同的地方做了不同的定义,那是一件更糟糕的事情。

4. 灵活性

用户喜欢可以灵活选择的软件,软件可以选择不同的状态和方式,完成相应的功能。但灵活性也可能发展为复杂性,太多的状态和方式的选择增加的不仅是用户理解和掌握的困难程度。多种状态之间的转换,增加了编程的难度,更增加了软件测试人员的工作量。图 7-4 中 Windows 计算器程序有两种方式:标准型和科学型,充分体现了灵活性。



图 7-4 Windows 计算器程序的灵活性

5. 舒适性

人们对舒适的理解各不相同,总体上说恰当的表现、合理的安排、必要的提示或更正能力等是要考虑的因素。如图 7-5 所示的状态信息可以让用户清楚目前的工作状态,就是一个很好的例子。

舒适性的例子比较多,例如,iPhone 手机为什么那么受欢迎,就是它的触摸屏操作非常方便、轻松。财务报表软件就不太适合绚丽的色彩、狂放的风格。Windows 的 UNDO/REDO 特性让用户觉得方便,左手鼠标的设置给惯用左手的人带来便利,也能为右手十分劳累时提供了另一种途径。

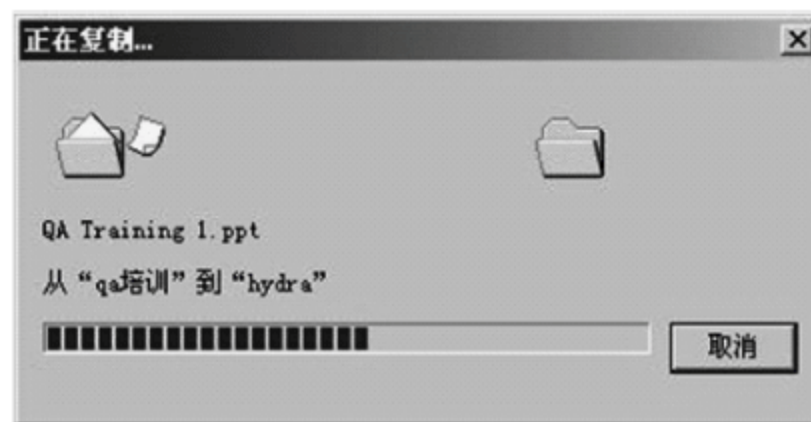


图 7-5 复制文件的状态

6. 正确性

正确性的问题一般都很明显,比较容易发现。通常,要注意是否有多余或遗漏的功能、功能是否被正确实现、语言拼写是否无误、在不同媒介上的表现是否一致、所有界面元素的状态是否都准确无误等。例如,根据用户的权限系统能否自动屏蔽某些功能、将密码输入内容是否显示为 * 号等。

7. 实用性

主要指软件产品的各个功能是否实用。在需求和产品规格说明书的评审、实际测试等过

程中都应考虑各个具体特性是否必要的、是否真正对用户具有实际价值。如果认为没有必要,就要研究或和产品设计人员讨论其存在的原因。无用的功能只会增加程序的复杂度,产生不必要的软件缺陷。

在大型软件的长期开发中或经过多个版本的演化过程中容易产生一些没有实用价值的功能,可能导致某个功能没有用了,或者原先设计界面上的图标或按钮没有存在的价值,也可能导致产生一些无用的数据等。

总而言之,软件的易用性没有一个具体量化的指标,主观性较强。当前面的7个要素处理好了,易用的属性自然就好了。界面清晰美观、各元素布置合理、符合常用软件的标准和规范、用户能够在不需要其他帮助的情况下完成各项主要功能的,就认为软件达到了易用性的要求。

7.4 安装测试和可恢复性测试

软件的安装测试是容易被忽略的一个环节,开发和测试人员均为专业人员,在开发和测试的过程和环境中容易忽略对非专业人员造成的问题。系统一旦开始运行以后,不能保证系统能无限期地正常工作下去,可能会出现系统宕机,需要及时恢复,这就要求进行系统的可恢复性测试,它是为了更好地保证软件的可用性和数据的可靠性。

1. 安装与卸载测试

软件产品的日益丰富,使可获得软件的途径也多种多样,软件的安装方式也发生了很大的变化,有客户端软件的安装、直接通过浏览器下载安装,还有服务器端的系统部署,随着软件即服务(Software as a Service)和云服务等,系统部署会越来越普遍,其中包括系统的升级、在系统运行时打补丁(Patch)等。客户端安装和服务器系统部署有很大差别,客户端安装测试时,主要验证能否安装成功、安装步骤是否清晰、中途是否退出、安装完之后能否顺利卸载、卸载时是否破坏用户数据、是否能正常升级等内容;而完整的系统部署会更复杂,需要考虑更多的因素,包括服务器架构、环境配置、数据备份等。安装测试时,一般要注意以下几个方面。

(1) 一般严格按照安装文档中的说明,一步一步地进行操作,最好从文档中复制出各种操作命令及其参数,确保输入到计算机的命令和文档中的内容一致。检查系统安装是否能够安装所有需要的文件、数据并完成必要的系统设置。

(2) 软件的安装说明书有无对安装环境做限制和要求。至少在标准配置和最低配置两种环境下安装。曾经有过这样的例子,某客户端产品进行安装测试时十分顺利,在准备发布之前的一次意外演示,发现安装无法通过,因为机器上没有Java虚拟机(Java Runtime Environment,JRE)。让经理主管们对测试结果产生了很大的疑问。真正的原因是测试用机在安装操作系统时默认会安装JRE,从而没有发现上述问题。

(3) 安装过程是否简单,容易掌握。软件的安装说明书与实际安装步骤是否一致。对一般用户而言,长长的安装文档、复杂的操作步骤往往造成畏惧心理。如果实际步骤与安装说明再有出入,就容易让用户缺乏信心,增加技术支持的成本。

(4) 安装过程是否有明显的、合理的提示信息,如选择/更改目录、安装的进程、下一步操作提示、中途退出(Cancel)等都应明确提示。如果是升级安装后,用户数据是否得到保护并能继续使用。

(5) 卸载测试也是安装测试的一部分。卸载后,文件、目录、快捷方式等是否清除,占用的系统资源是否全部释放、是否影响其他软件的使用。更重要的是,用户数据是否被保留,不能

被删除,如果要删除,也需要提示用户,由用户做出选择。

(6) 安装过程中是否会出现不可预见的或不可修复的错误,进一步验证安装过程中(特别是系统软件)对硬件的识别能力、是否会破坏其他文件或配置。

(7) 安装程序是否占用太多系统资源、是否有冲突、是否会影响原系统的安全性。

(8) 软件安装的完整性和灵活性。大型的应用程序会提供多种安装模式(最大、最小、自定义等),每种模式是否能够正确地执行,以及是否可以中止并恢复现场。

(9) 软件使用的许可号码或注册号码的验证,用户许可协议的条款要保证其合理、合法。

2. 可恢复性测试

提供软件服务的计算机系统必须在限定的时间内从失效状态中恢复过来,最大限度地减少对服务的影响。有些软件系统具有很好的容错性,也就是说,运行过程中出现的错误对局部有影响,不能造成整个系统的崩溃。在某些情况下,一旦某个子系统出现问题,由一个备份子系统将服务接替过来,从而不会影响整个系统。如果这个系统出了问题,由一个备份系统完全将服务接受过来,继续提供服务。

恢复测试主要检查系统的容错能力。当系统出错时,能否在指定时间间隔内修正错误或重新启动系统。恢复测试首先要通过各种手段,让软件强制性地发生故障,然后验证系统是否能尽快恢复。对于自动恢复需验证重新初始化、检查点、数据恢复和重新启动等机制的正确性;对于人工干预的恢复系统,还需评估平均修复时间,确定其是否在可接受的范围内。例如,基于客户/服务器结构的应用是测试工作中常常遇到的,下面就是一个简单的示例,对我们有所启发。

先分析服务器端的恢复测试,通常服务器上会有一个进程是对其他服务进程进行维护和管理。本例是一台 Linux 系统的服务器。使用“pgrep -flsvr”命令列出所有服务进程如下所示,其中“atmmsvr”为维护管理进程,其他均为各种服务进程。

```
[root@lnx2210 root]# pgrep -flsvr
12063 /opt/... .. /ammsvr
12137 /opt/... .. /apngsvr 192.168.2.211
12138 /opt/... .. /acb1svr 192.168.2.213
12139 /opt/... .. /acb2svr 192.168.2.214
12140 /opt/... .. /arassvr 192.168.2.215
12141 /opt/... .. /alicsvr 192.168.2.212
12142 /opt/... .. /alogsvr 192.168.2.212
12143 /opt/... .. /achatsvr 192.168.2.213
12144 /opt/... .. /aassvr 192.168.2.213
12145 /opt/... .. /adtsvr 192.168.2.213
12146 /opt/... .. /achatsvr 192.168.2.214
12147 /opt/... .. /aassvr 192.168.2.214
12148 /opt/... .. /adtsvr 192.168.2.214
... ..
12290 /opt/... .. /wmssvr
12378 /opt/... .. /arassvr 192.168.2.215
12592 /opt/... .. /apngsvr 192.168.2.211
12593 /opt/... .. /apngsvr 192.168.2.211
[root@lnx2210 root]# kill -9 12138
```

如果对其中进程号为 12138 的“acb1svr”进行恢复测试,可以使用“kill -9 12138”命令将该

进程杀掉。立刻通过客户端验证该项服务的丧失,在恢复时间内监控服务器的进程,直到“acblsvr”进程被重新启动。再通过客户端验证该项服务的恢复。服务器端系统资源不应该出现较大的变化。

再分析客户端的恢复测试,可以用一个更简单的例子说明。手工拔下网线,在许可的时间范围内再插上。从客户的角度,服务的丢失和重新获得不能太麻烦,也不能太困难,状态不能发生大的变化,数据能够重新获得。

小结

验收测试是技术测试的最后一个阶段,需要和用户共同完成,而且需要在软件实际运行环境中进行测试。严格按照产品规格说明书来进行测试,通过验收测试后,需要提交验收测试报告。在本章还详细介绍了安装测试、可恢复性和文档测试等,安装测试可以扩展到数据中心的部署验证,而文档测试主要检查文档的正确性、完备性、易理解性和一致性。

思考题

1. 验收测试是由用户完成的吗? 为什么?
2. 进行验收测试的条件是什么? 通过标准是什么?
3. 如何进行产品说明书的验证?
4. 简述文档测试的重要性。
5. 用户界面测试有哪些要素?
6. 软件测试分为哪 4 个阶段? 每个阶段的主要任务和目标是什么?